

RCMultiBus – Übersicht und Funktionsweise

Der **RCMultiBus** dient als übergeordnetes Protokoll für alle 3-Draht-basierten Datenbusse, die im RC-Bereich verwendet werden. Ein Slave-Gerät, das RCMultiBus unterstützt, muss in der Lage sein, alle Bus-Typen inklusive Protokoll, Baudrate und Polarität zu interpretieren. Ein Master-Gerät hingegen muss nur einen der Bus-Typen erzeugen können.

Unterstützte Bus-Typen

- IBUS
- SBUS
- EX-Bus
- CPPM

Je nach Bus-Typ können proportionale und/oder digitale Informationen empfangen werden. Jeder proportionale Kanal wird stets in digitale Schaltfunktionen (permanent und im Speicher) umgewandelt.

Kanaldaten und Bewegungsüberwachung

Jeder Kanalwert wird gespeichert und kann für jeden Servo- oder Motor-Ausgang individuell zugeordnet werden. Zusätzlich wird die Bewegung jedes Kanals überwacht, woraus ein Bitfeld generiert wird. Dieses Bitfeld hat drei Dimensionen:

- **Richtung:** oben (top) / unten (bottom)
- **Bewegungsintensität:** voll (full) / halb (half)
- **Status:** permanent / im Speicher (memory)

CH	Memory Short	Memory Long	Permanent	Top Full	Top Half	Bottom Half	Bottom Full
CH1	Memory Short	Memory Long	Permanent	Top Full	Top Half	Bottom Half	Bottom Full
CH2	Memory Short	Memory Long	Permanent	Top Full	Top Half	Bottom Half	Bottom Full
...							
CHx	Memory Short	Memory Long	Permanent	Top Full	Top Half	Bottom Half	Bottom Full

Beispiele zur Bewegungsüberwachung

- **Beispiel 1:** Kanal 4 wird auf +30% bewegt → Flag *CH4 Permanent/Top/Half* wird gesetzt.
- **Beispiel 2:** Kanal 4 wird innerhalb von 0,5 Sekunden zurückbewegt → Flag *CH4 Permanent/Top/Half* wird gelöscht, Flag *CH4 Memory/Top/Half* wird gesetzt.
- **Beispiel 3:** Kanal 1 wird auf -100% bewegt → Flag *CH1 Permanent/Bottom/Full* wird gesetzt.
- **Beispiel 4:** Kanal 1 wird nach einer Sekunde zurückbewegt → Flag *CH1 Permanent/Bottom/Full* wird gelöscht, Flag *CH1 Memory/Bottom/Full* wird gesetzt.

Multiswitch-Systeme (Futaba, Graupner)

Multiswitch-Systeme sind langsamer, da die Signale multiplexartig nacheinander übertragen werden und hauptsächlich Knöpfe oder Schalter (keine Sticks) verwenden. Daher können die Zustände „short“ und „half“ nicht verwendet werden.

CH	Memory Short	Memory Long	Permanent	Top Full	Top Half	Bottom Half	Bottom Full
CH1	Memory Short	Memory Long	Permanent	Top Full	Top Half	Bottom Half	Bottom Full
CH2	Memory Short	Memory Long	Permanent	Top Full	Top Half	Bottom Half	Bottom Full
...							
CHx	Memory Short	Memory Long	Permanent	Top Full	Top Half	Bottom Half	Bottom Full

SUMD und SUMDV3

Ob SUMD oder SUMDV3 verwendet wird, hängt von der Version des Senders/Empfängers und der Firmware des Empfängers ab.

- **SUMD:** Funktioniert analog zu IBUS, SBUS, EX-Bus und CPPM (siehe oben).
- **SUMDV3:** Überträgt ein Feld sogenannter „digitaler Schalter“, die von RCMultiBus-Modulen empfangen und gespeichert werden. Hier können sowohl kanalspezifische Schalter (wie bei IBUS, SBUS, EX-Bus, CPPM) als auch SUMDV3-spezifische digitale Schalter verwendet werden.

PWM Standard

Funktioniert ähnlich wie IBUS, SBUS, EX-Bus und CPPM, allerdings mit nur einem Kanal (CH1).

UART-Einstellungen

- Aktuelle Einstellung: 38.400 Baud, 8 Bytes, LSB zuerst, keine Parität, 1 Stopbit, Timeout 100 µs
- Zukünftig geplant: 100.000 Baud

Zentrale zeitgesteuerte Funktionen

Zentrale zeitgesteuerte Funktionen wie Blinker werden durch zwei digitale Funktionen gesteuert:

- Information, ob die Funktion generell ein- oder ausgeschaltet ist
- Impuls, der die Lampe an- und ausschaltet

Um den Blinker zu steuern, reicht es aus, die Impulsinformation zu setzen. Die andere Information kann für Mixer und weitere Schalter verwendet werden, z. B. um bei eingeschaltetem Blinker auf eine zweite Konfiguration umzuschalten.

Datenstrukturen

Basisfunktionen (Tag 0x01)

Die Basisfunktionen sind in 5 Bytes codiert:

Offset	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
0	Wamblinker	Blinker links	Blinker rechts	Fernlichtblitzer	Nebelscheinwerfer	Fernlicht	Abblendlicht	Standlicht
1	Stützbeine hoch	Fünfte Rad	Motor läuft	Anlasser	Hupe	Hecknebellicht	Bremse	Rückwärtsgang
2	Blinker rechts Impuls	Rundumleuchte	Tagfahrlicht	Blinker links	Senken	Heben	Stützbeine runter	
3	Intern/RFU	Signalhupe	Gang runter	Gang rauf	Kurvenlicht links	Kurvenlicht rechts	Intern/RFU	Blinker links Impuls
4	Servo 2	Servo 1	Gang 2/2	Gang 1/2	Pad Level	Rampe runter	Rampe hoch	Intern/RFU

Zusatzfunktionen (Tag 0x05)

Offset	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
0	CH1 Memory Half Short Rechts	CH1 Memory Half Short Links	CH1 Rechts Voll	CH1 Links Voll	CH1 Memory Voll Lang Rechts	CH1 Memory Voll Lang Links	CH1 Memory Voll Kurz Rechts	CH1 Memory Voll Kurz Links
1	CH2 Memory Voll Lang Rechts	CH2 Memory Voll Lang Links	CH2 Memory Voll Kurz Rechts	CH2 Memory Voll Kurz Links	CH1 Rechts Halb	CH1 Links Halb	CH1 Memory Halb Lang Rechts	CH1 Memory Halb Lang Links
2	CH2 Rechts Halb	CH2 Links Halb	CH2 Memory Halb Lang Rechts	CH2 Memory Halb Lang Links	CH2 Memory Halb Kurz Rechts	CH2 Memory Halb Kurz Links	CH2 Rechts Voll	CH2 Links Voll
...								

Offset	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
45	CH31 Memory Halb Kurz Rechts	CH31 Memory Halb Kurz Links	CH31 Rechts Voll	CH31 Links Voll	CH31 Memory Voll Lang Rechts	CH31 Memory Voll Lang Links	CH31 Memory Voll Kurz Rechts	CH31 Memory Voll Kurz Links
46	CH32 Memory Voll Lang Rechts	CH32 Memory Voll Lang Links	CH32 Memory Voll Kurz Rechts	CH32 Memory Voll Kurz Links	CH31 Rechts Halb	CH31 Links Halb	CH31 Memory Halb Lang Rechts	CH31 Memory Halb Lang Links
47	CH32 Rechts Halb	CH32 Links Halb	CH32 Memory Halb Lang Rechts	CH32 Memory Halb Lang Links	CH32 Memory Halb Kurz Rechts	CH32 Memory Halb Kurz Links	CH32 Rechts Voll	CH32 Links Voll

Proportionale Funktionen (Tag 0x41)

Die Werte sind 16-Bit signed Werte von -500 bis +500, was einer Anzeige von -100 % bis +100 % entspricht. Die Daten sind im Little-Endian-Format gespeichert.

CH1 (Gas)	CH2 (Lenkung)	CH3	CH4	...	CH31	CH32
-----------	---------------	-----	-----	-----	------	------

Befehle

Alle Befehle beginnen mit 0xA1 oder 0xA2, sind im Tag-Length-Value-Format (TLV) kodiert und enden mit einer XOR-Prüfsumme.

Startbyte	Tag (Befehl)	Länge	Wert (Daten)	Prüfsumme
0xA1 / 0xA2	0x6C	0x03	0x01 0x01 0x01	0xCF

Die Prüfsumme wird als XOR über alle Bytes mit Startwert 0xAA berechnet.

SET_FCT_DIRECT

Mit der Funktion *SetFunctionDirect* können einzelne Funktionen direkt ein- oder ausgeschaltet werden.

Startbyte	Tag (Befehl)	Länge	Wert (Daten)	Prüfsumme
0xA1	0x6C	0x03	0x01 0x03 0x01	0xCD

Datenformat

Byte 1	Byte 2	Byte 3
Datenobjekt (z. B. Basisfunktionen)	Bitoffset 1–40 (z. B. Parklicht, Abblendlicht)	Ein/Aus (1 = Ein, 0 = Aus)

SET_PARAMETER

Mit der Funktion *SetParameter* können Skriptparameter für alle Ausgänge geändert werden, z. B. „Alle Fernlicht-Helligkeitswerte auf 80 % setzen“.

Startbyte	Tag (Befehl)	Länge	Wert (Daten)	Prüfsumme
0xA1	0x76	0x02	0x04 0x50	0x81

SET_DATA

Diese Funktion dient dazu, periodisch alle Werte – sowohl proportionale als auch digitale Funktionen – zu setzen.

Startbyte	Tag der Datenstruktur	Länge	Wert (Daten)	Prüfsumme
0xA2	Tag der Datenstruktur	0x05	0x87 0x80 0x00 0x00 0x00	0xCF

KRAFTWERK

image1.png

RCMultiBus

image2.png